

DTS103TC

Data Structure and Algorithms

Lab 4: Binary Search Tree and Balanced Trees

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcome

- Binary Search Tree
- AVL Tree

Binary Search Tree

Binary Search Tree (BST) Definitions

- A binary search tree is a binary tree where each node has a key
- The key in the left child (if exists) of a node is less than (or equal to) the key in the parent
- The key in the right child (if exists) of a node is greater than (or equal to) the key in the parent
- The left & right subtrees of the root are again binary search trees.

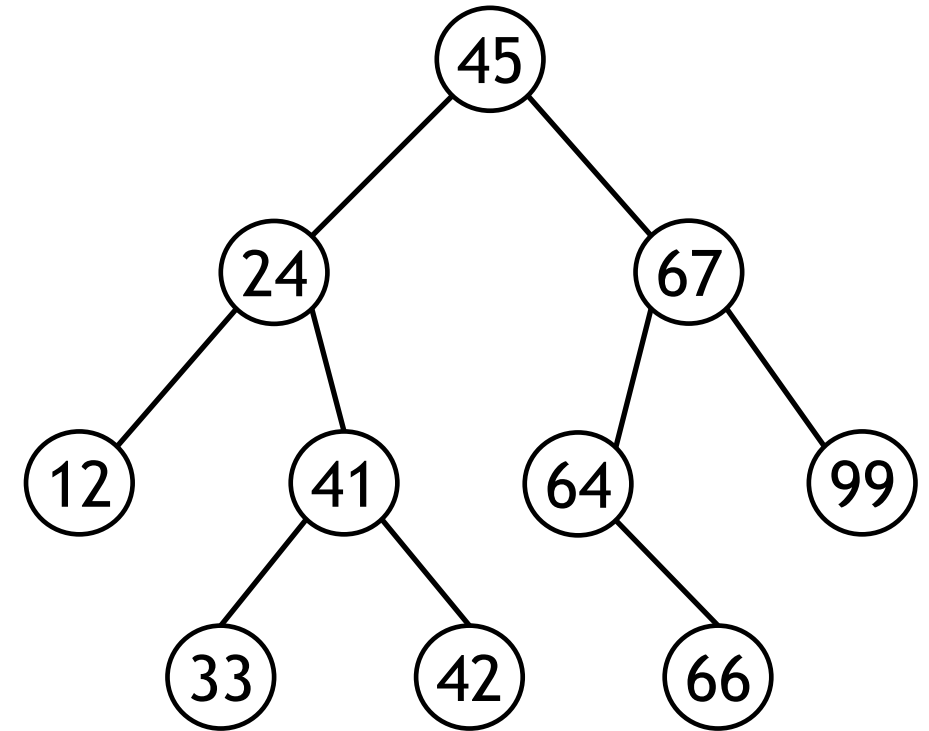
Binary Search Tree

- Each Node has
 - Left
 - Right
 - Key
- For each node n , with key k
 - $n.left$ contains only nodes with keys $\leq k$
 - $n.right$ contains only nodes with keys $\geq k$
- $O(\log n)$ on average for insert, lookup, and remove.

```
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
```

Inorder Traversal

```
inorderTraversal(x)
If x ≠ NULL
    inorderTraversal(x.left)
    print (x.key)
    inorderTraversal(x.right)
```



[12, 24, 33, 41, 42, 45, 64, 66, 67, 99]

Preorder Traversal: Recursive

- Process (display) the value in the node
- Traverse the left subtree preorder
- Traverse the right subtree preorder

```
preorderTraversal(x)
If x ≠ NULL
    print (x.key)
    preorderTraversal(x.left)
    preorderTraversal(x.right)
```

Postorder Traversal: Recursive

- Traverse the left subtree postorder
- Traverse the right subtree postorder
- Process (display) the value in the node

```
postorderTraversal(x)
If x ≠ NULL
    postorderTraversal(x.left)
    postorderTraversal(x.right)
    print (x.key)
```

Searching

- Given a pointer to the root of the tree and a key, return a pointer to a node with key k if one exists; otherwise, return NULL

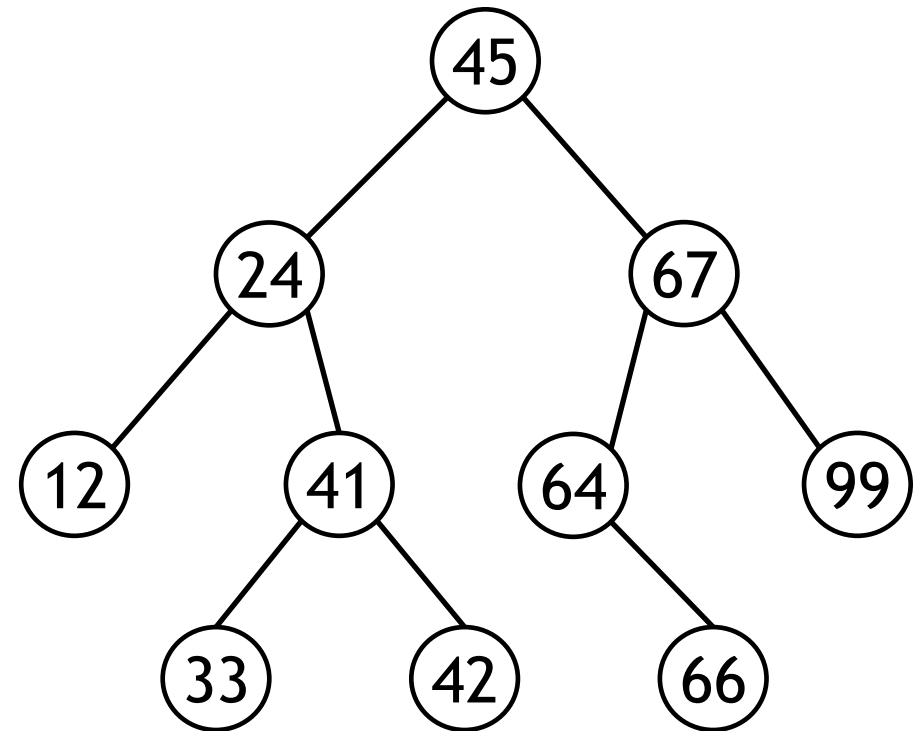
```
Search(x,k)
if x == NULL or k == x.key
    return x
if k < x.key
    return Search(x.left,k)
else
    return Search(x.right,k)
```

```
Search(x,k)
while x ≠ NULL and k ≠ x.key
    if k < x.key
        x = x.left
    else
        x = x.right
return x
```

Minimum and Maximum

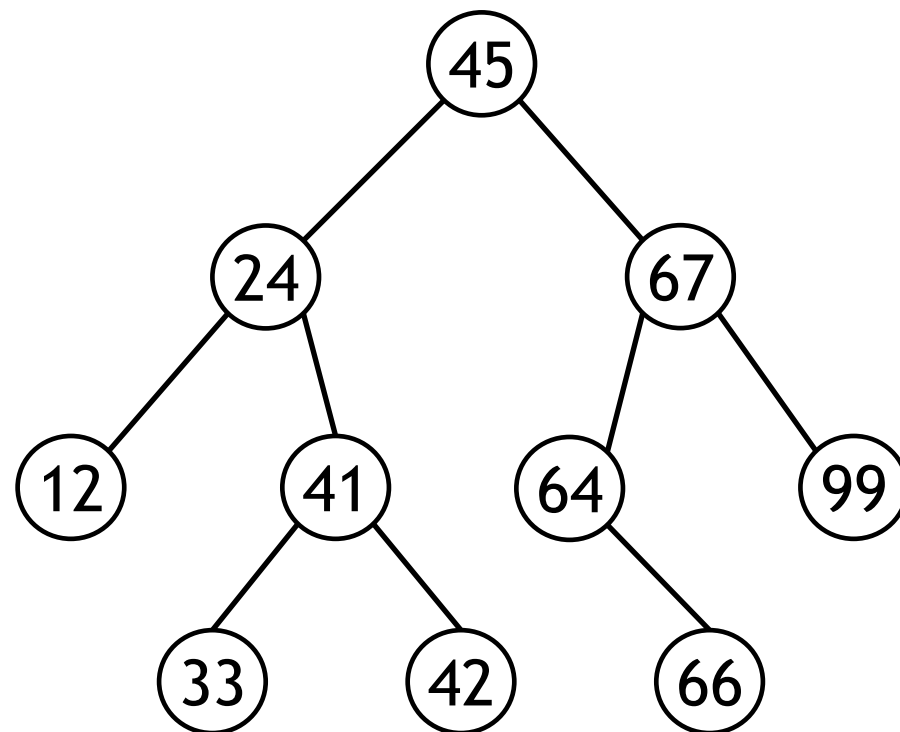
```
Minimum(x)  
while x.left ≠ NULL  
    x = x.left  
return x
```

```
Maximum(x)  
while x.right ≠ NULL  
    x = x.right  
return x
```



Insertion

```
Insert(T,z)
y=NULL
x=T.root
While x ≠ NULL
    y=x
    if z.key < x.key
        x = x.left
    else
        x = x.right
z.parent = y
if y == NULL
    T.root = z
elseif z.key < y.key
    y.left = z
else
    y.right = z
```



Insert 65 ?
Insert 32 ?

Deletion

- The overall strategy for deleting a node z from a binary search tree T has three basic cases
 - If z has no children, then we simply remove it by modifying its parent to replace z with NULL as its child.
 - If z has just one child, then elevate that child to take z 's position in the tree by modifying z 's parent to replace z by z 's child.
 - If z has two child, then we find z 's successor y and have y take z 's position in the tree.

Deletion

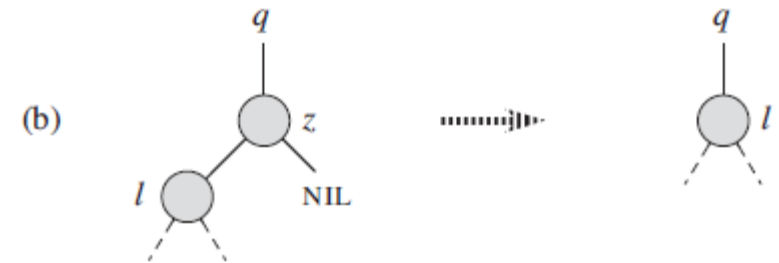
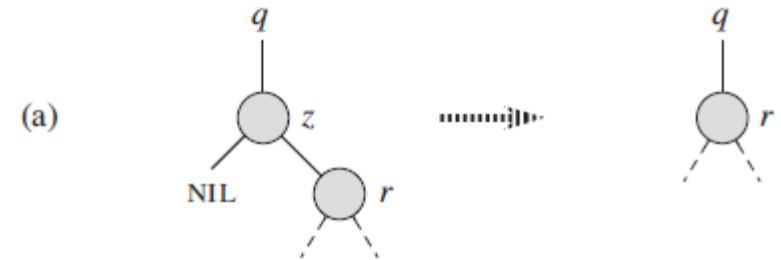
- To move subtrees around within the binary search tree, we define a function Transplant to replace one subtree as a child of its parent with another subtree.
- Replace the subtree rooted at node u with the subtree rooted at node v .

```
Transplant(T,u,v)
if u.parent = NULL
    T.root = v
elseif u==u.parent.left
    u.parent.left = v
else
    u.parent.right = v
if v ≠ NULL
    v.parent = u.parent
```

Deletion

- Delete node z from binary search Tree T :

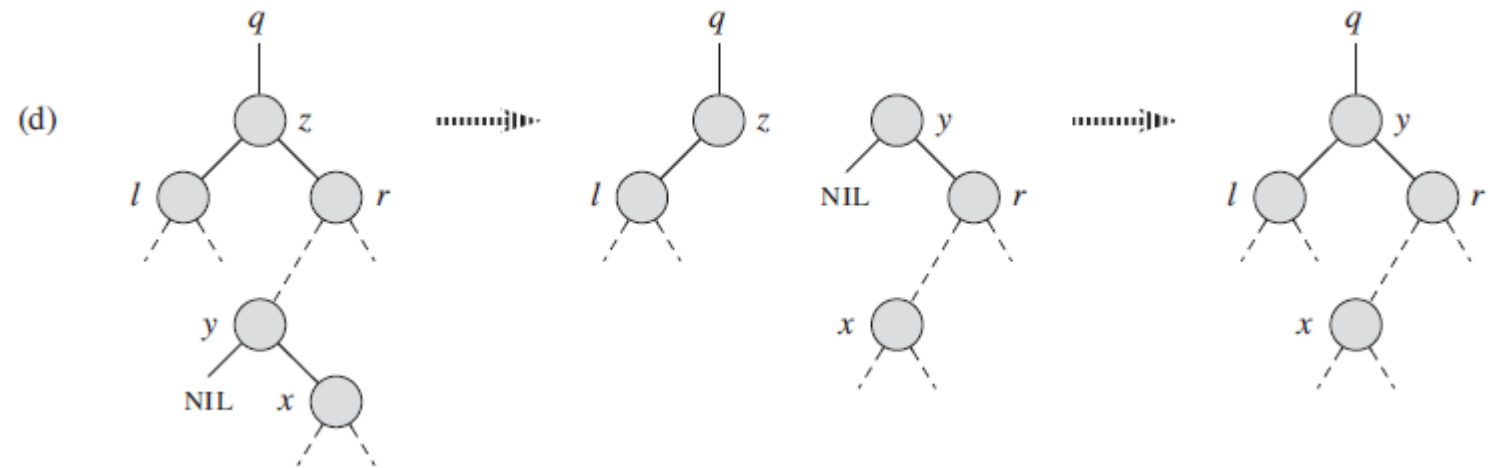
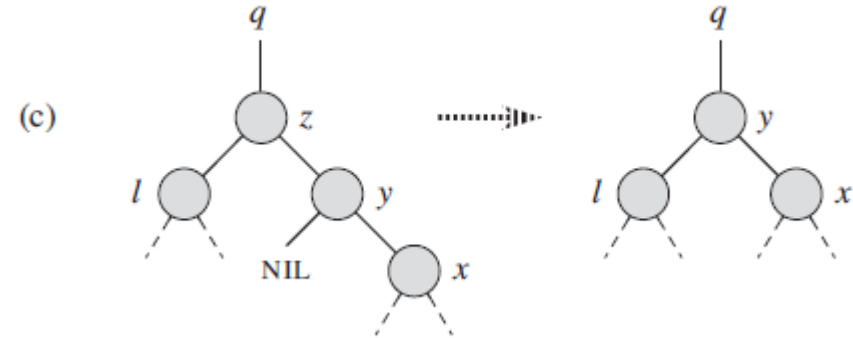
```
Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
```



Deletion

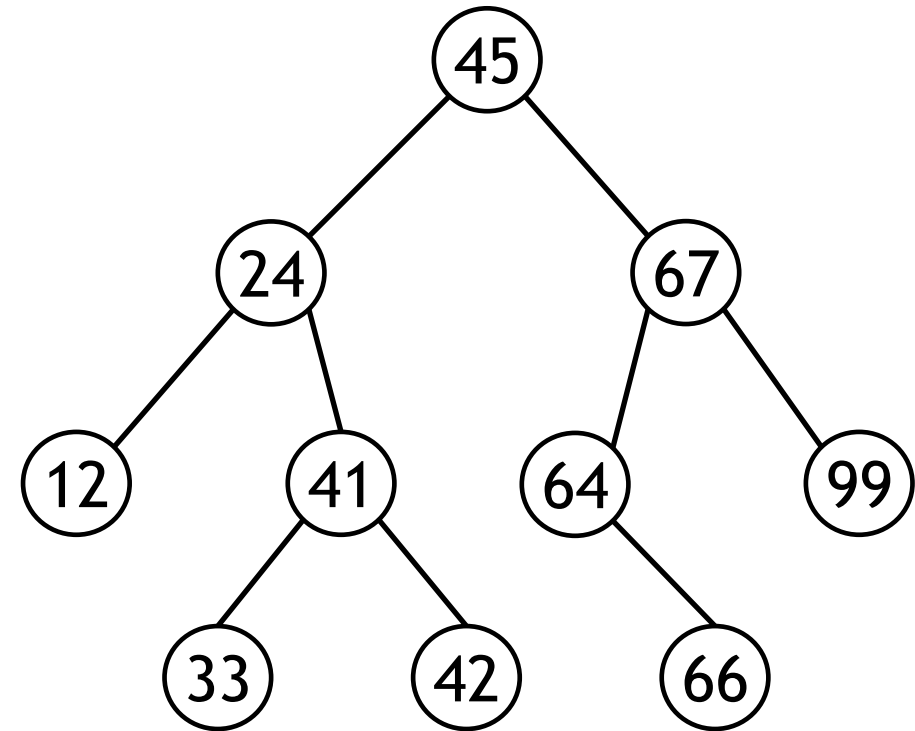
```

Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
    
```



Exercise: Deletion

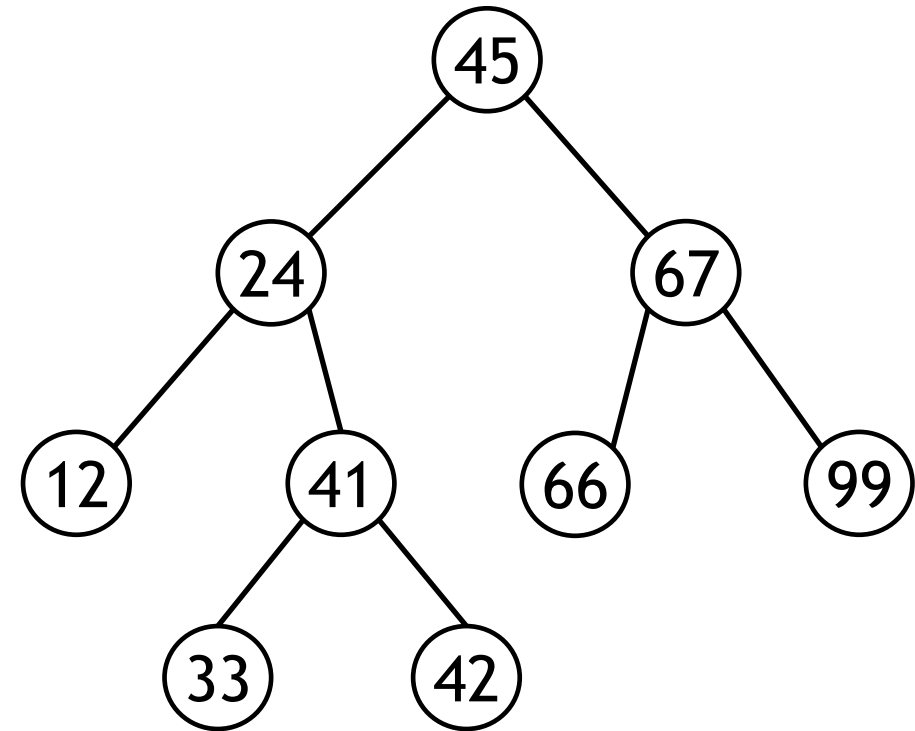
```
Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
```



Delete 64 ?

Exercise: Deletion

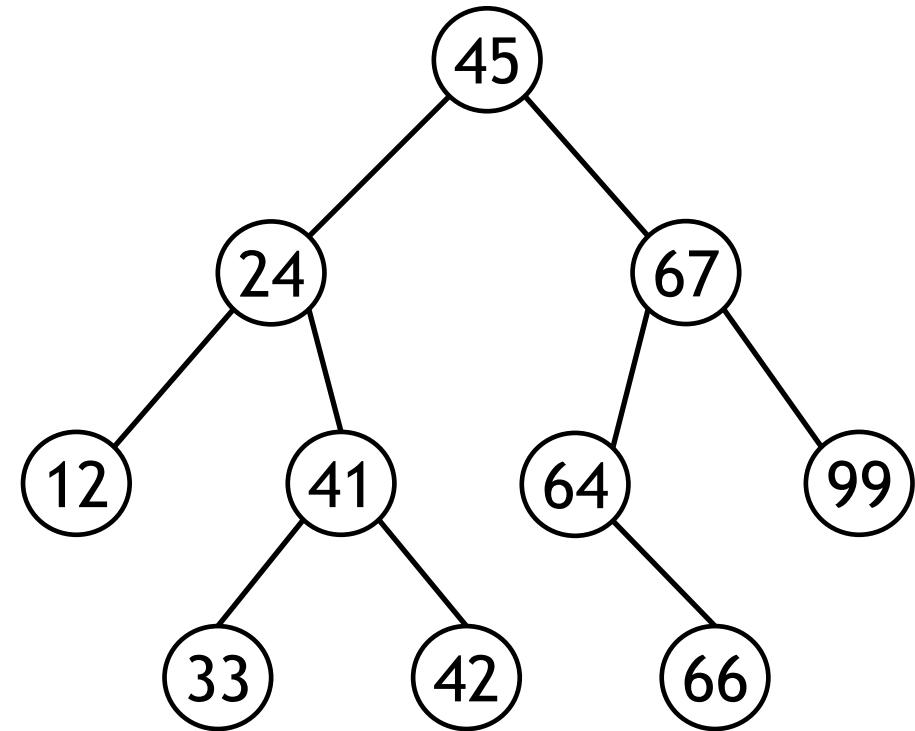
```
Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
```



After deletion

Exercise: Deletion

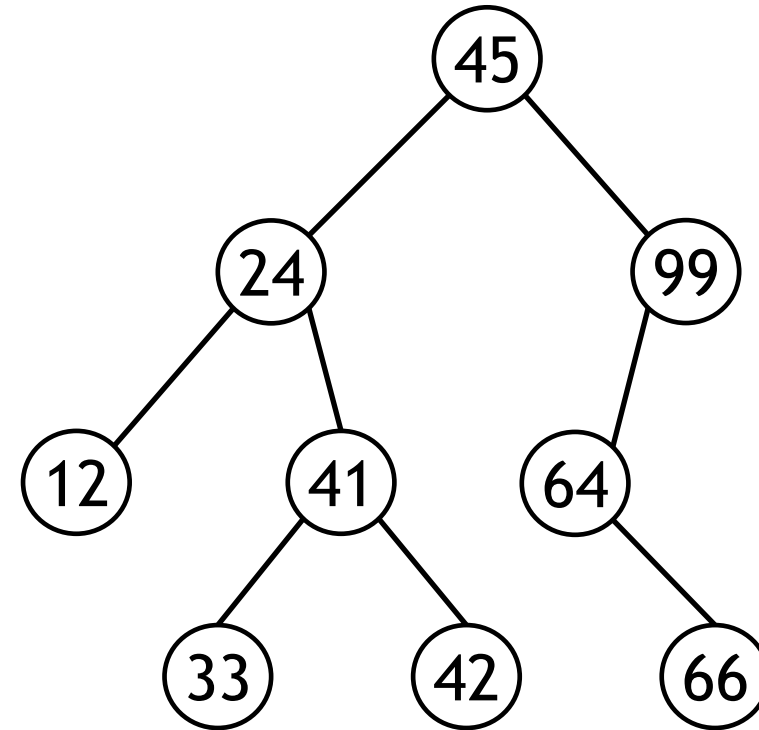
```
Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
```



Delete 67 ?

Exercise: Deletion

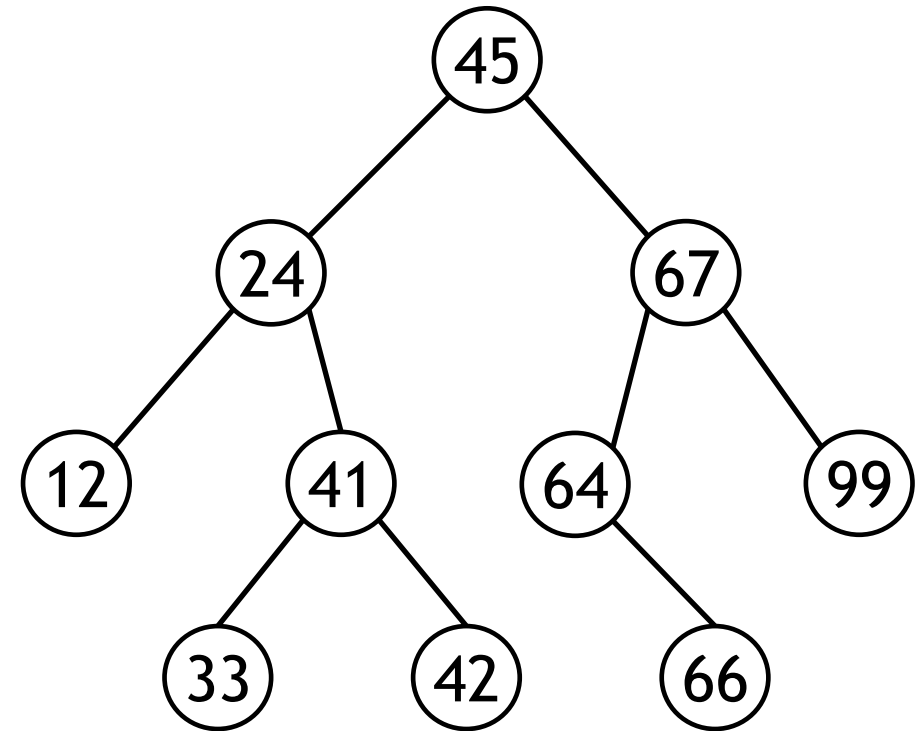
```
Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
```



After deletion

Exercise: Deletion

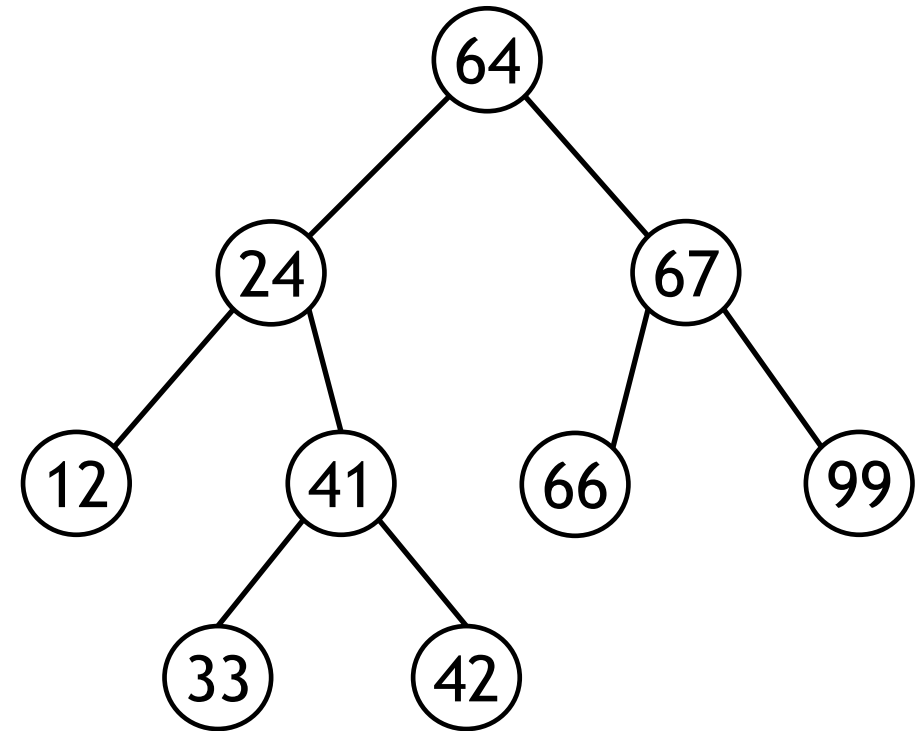
```
Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
```



Delete 45?

Exercise: Deletion

```
Delete(T,z)
If z.left == NULL
    Transplant(T,z,z.right)
elseif z.right == NULL
    Transplant(T,z,z.left)
else
    y = minimum(z.right)
    if y.parent != z
        Transplant(T,y,y.right)
        y.right = z.right
        y.right.parent = y
    Transplant(T,z,y)
    y.left = z.left
    y.left.parent = y
```



After deletion

Lab 4

- Binary Search Tree
 - Inorder Traversal
 - Insert
 - Search
 - Minimum
 - Delete

AVL Tree

AVL Tree

- AVL tree got its name after its inventor Adelson-Velsky and Landis.
- AVL tree is height-balanced binary search tree
- Balance factor of a node
 - Balance Factor = (Height of Left Subtree - Height of Right Subtree)
 - The self balancing property of an AVL tree is maintained by the balance factor.
- The value of balance factor should always be -1, 0 or +1.

AVL Tree

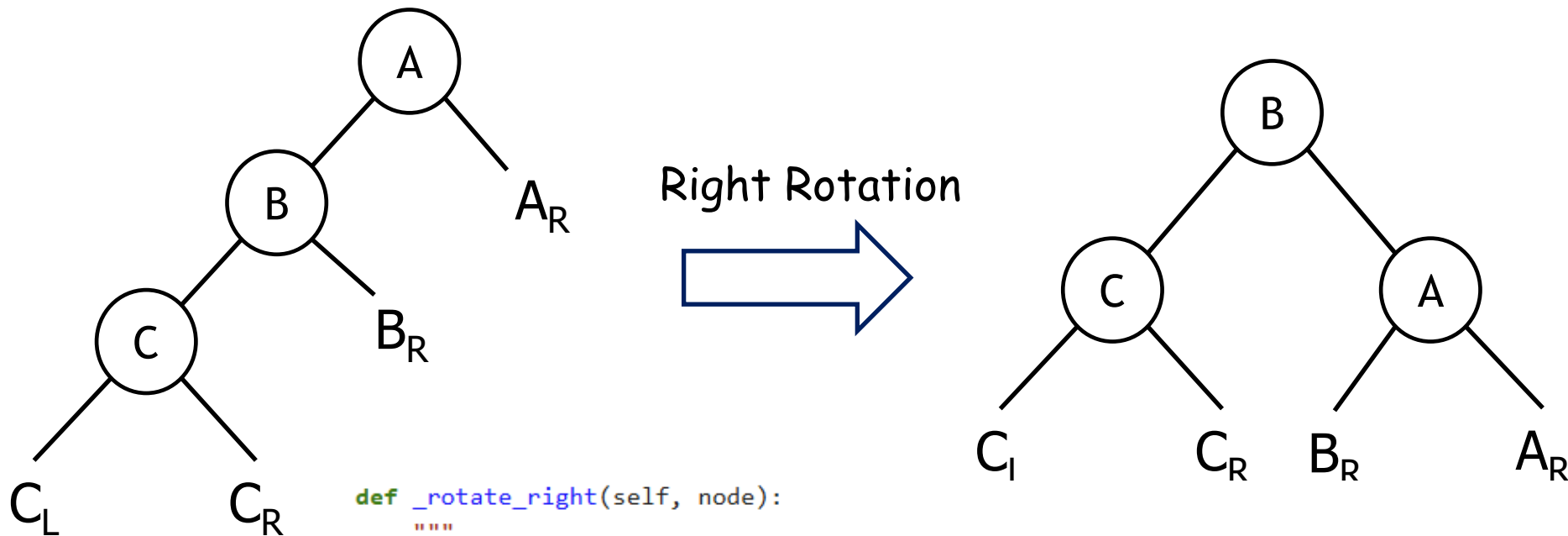
- Each Node has
 - Left
 - Right
 - Key
 - Height
- Helper Functions
 - `_get_height(node)`
 - `_get_balance(node)`

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

    def _get_height(self, node):
        # Return node height, or 0 if node is None
        if not node:
            return 0
        return node.height

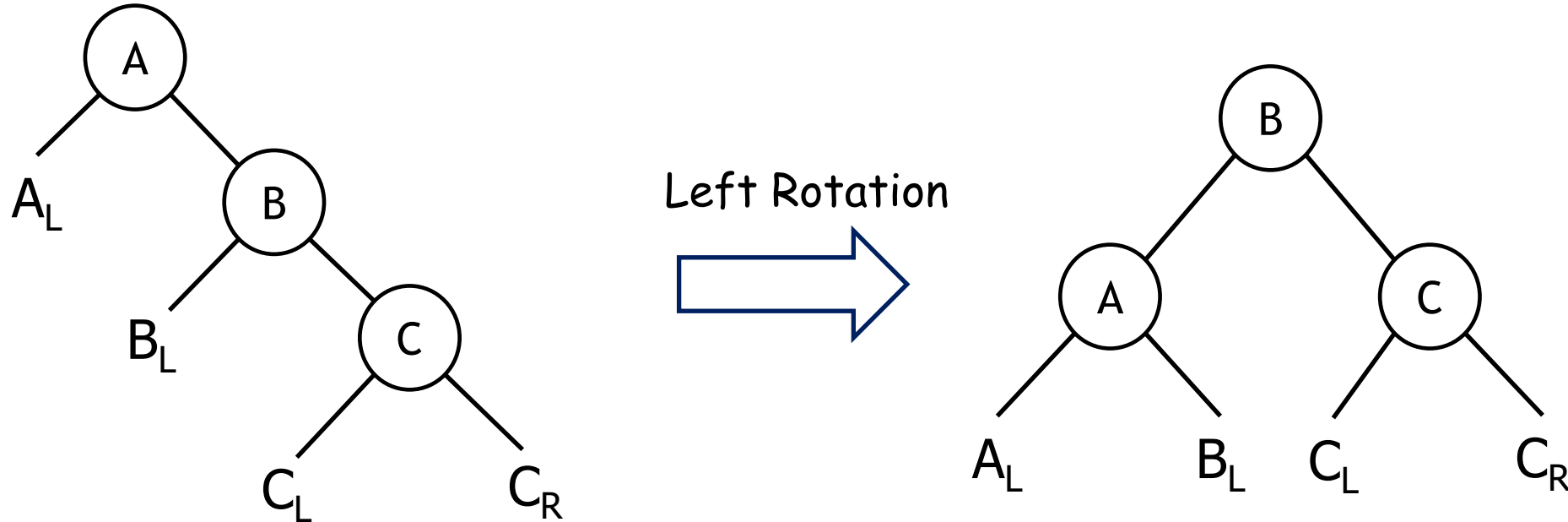
    def _get_balance(self, node):
        # Balance factor = height(left subtree) - height(right subtree)
        if not node:
            return 0
        return self._get_height(node.left) - self._get_height(node.right)
```

Rotations



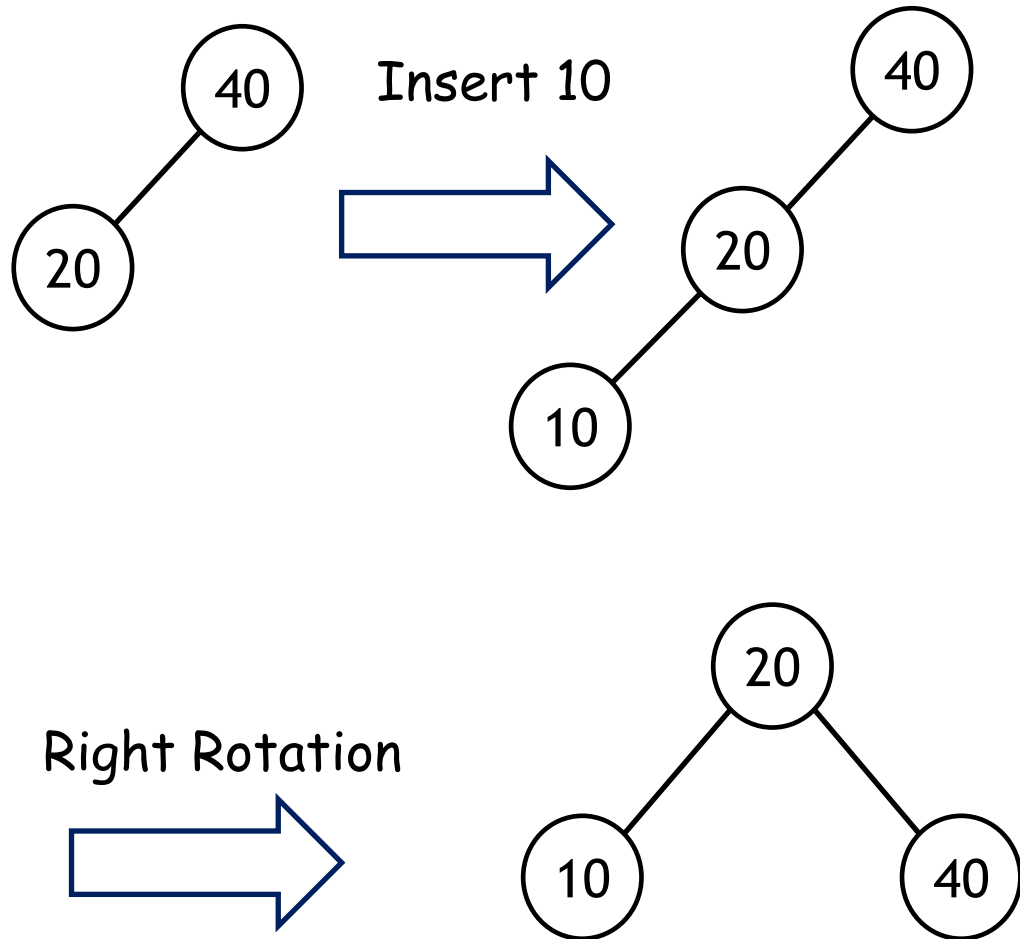
```
def _rotate_right(self, node):  
    """  
    Perform a right rotation and return the new root of this subtree.  
    """  
    new_root = node.left  
    node.left = new_root.right  
    new_root.right = node  
  
    # Update heights after rotation  
    node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))  
    new_root.height = 1 + max(self._get_height(new_root.left), self._get_height(new_root.right))  
  
    return new_root
```

Rotations



```
def _rotate_left(self, node):  
    """  
    Perform a left rotation and return the new root of this subtree.  
    """  
    new_root = node.right  
    node.right = new_root.left  
    new_root.left = node  
  
    # Update heights after rotation  
    node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))  
    new_root.height = 1 + max(self._get_height(new_root.left), self._get_height(new_root.right))  
  
    return new_root
```

Insertion Example



```
def insert(self, key):
    # Public method: insert a key into the AVL tree
    self.root = self._insert_helper(self.root, key)

def _insert_helper(self, node, key):
    """
    Insert a key recursively and rebalance the tree if needed.
    """
    # Standard BST insertion
    if not node:
        return Node(key)
    elif key < node.key:
        node.left = self._insert_helper(node.left, key)
    else:
        node.right = self._insert_helper(node.right, key)

    # Update the height of the current node
    node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))

    # Calculate the balance factor
    balance_factor = self._get_balance(node)

    # Case 1: Left Left
    if balance_factor > 1 and key < node.left.key:
        return self._rotate_right(node)

    # Case 2: Right Right
    if balance_factor < -1 and key > node.right.key:
        return self._rotate_left(node)

    # Case 3: Left Right
    if balance_factor > 1 and key > node.left.key:
        node.left = self._rotate_left(node.left)
        return self._rotate_right(node)

    # Case 4: Right Left
    if balance_factor < -1 and key < node.right.key:
        node.right = self._rotate_right(node.right)
        return self._rotate_left(node)

    return node
```

Lab 4

- AVL Tree
 - Rotation
 - Insertion
 - Deletion